

Summer of Science 2026: Midterm Report

Topic: Neural Networks & Deep Learning

Dhruv Singla

Roll No: 24B1211

Mentor: Alok Kale

June 22, 2026

Contents

1	Introduction	3
2	Week 1: Core Concepts & Single Neuron Math	3
2.1	The Anatomy of a Forward Pass	3
2.2	Building Micrograd: The Value Object	4
3	Week 2: Backpropagation & Optimization Math	7
3.1	The Chain Rule and Gradients	7
3.2	Directed Acyclic Graph (DOT-style Representation)	7
3.3	Automating the Backward Pass (Topological Sort)	7
4	Week 3: Evaluation and Implementation Basics	9
4.1	Bias, Variance, and Data Splitting	9
4.2	Evaluation Metrics & Loss Functions	9
5	Week 4: Ground-Up Implementation (Midterm Project)	10
5.1	MLP Architecture Design	10
5.2	Translating Math to Matrix Operations in NumPy	10
5.3	The Backward Pass & L2 Regularization	11
5.4	Stabilizing Training with Batch Normalization	12
5.5	Midterm Results	12
6	Conclusion	13

1 Introduction

This report documents my progress over the first four weeks of the Summer of Science 2026 program. The core focus of my study has been "Neural Networks & Deep Learning," moving from the absolute fundamentals of single-neuron mathematics to building complete, functioning networks from scratch.

My learning approach has been highly iterative and hands-on. Instead of jumping straight into high-level libraries like PyTorch or TensorFlow, I spent these first four weeks intentionally reinventing the wheel. By following Andrej Karpathy's "Zero to Hero" lectures alongside StatQuest and 3Blue1Brown for visual intuition, I focused on understanding the actual calculus and matrix algebra that makes deep learning work.

I wanted to understand exactly what happens under the hood when we call `.backward()` or `.step()`. To do this, I built a micro-framework from the ground up, manually calculated forward and backward passes on paper, and then scaled up to a pure-NumPy implementation of a Multi-Layer Perceptron (MLP) complete with regularization and batch normalization.

2 Week 1: Core Concepts & Single Neuron Math

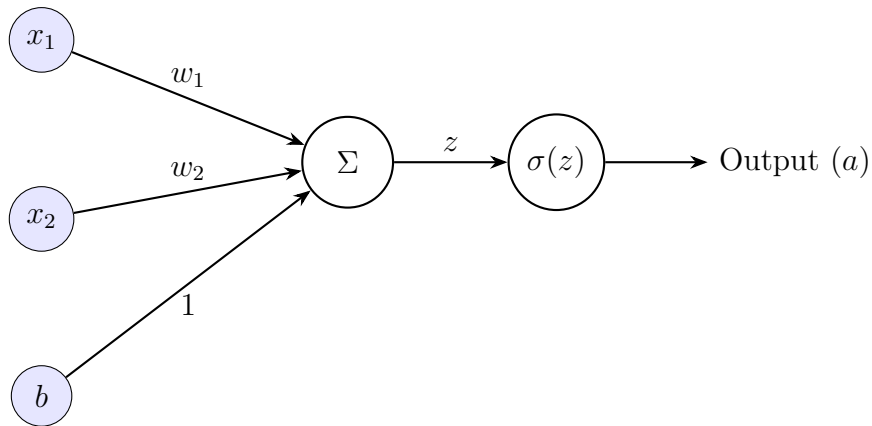
2.1 The Anatomy of a Forward Pass

When I first started looking at neural networks, the jargon made them sound like black magic. But when you strip it down, a neuron is just a container for a very basic mathematical operation: a dot product followed by a non-linear squash.

For my first actionable item, I sat down and calculated a basic forward pass on paper to understand exactly what weights and biases do. Suppose we have a neuron with three inputs. We assign weights to these inputs, multiply them, add them all up, and throw in a bias term. The bias is essentially a threshold that determines how "happy" the neuron needs to be before it fires.

$$z = \sum_{i=1}^n (w_i \cdot x_i) + b \tag{1}$$

After computing z , we pass it through an activation function. I started with the `tanh` function because it nicely bounds the output between -1 and 1, keeping the numbers manageable.



2.2 Building Micrograd: The Value Object

To truly understand how deep learning libraries track gradients, I followed Karpathy's method of building a scalar automatic differentiation engine. I created a `Value` class in Python. This class wraps standard Python floats, but it remembers the mathematical operations that created it. This is crucial because, to perform backpropagation later, the code needs to know the "ancestry" of every number.

Here is the foundational code I wrote for the Value object:

```
1 import math
2
3 class Value:
4     def __init__(self, data, _children=(), _op='', label=''):
5         self.data = data
6         self.grad = 0.0
7         self._backward = lambda: None
8         self._prev = set(_children)
9         self._op = _op
10        self.label = label
11
12    def __repr__(self):
13        return f"Value(data={self.data})"
14
15    def __add__(self, other):
16        other = other if isinstance(other, Value) else Value(other)
17        out = Value(self.data + other.data, (self, other), '+')
18
19        def _backward():
20            self.grad += 1.0 * out.grad
21            other.grad += 1.0 * out.grad
22        out._backward = _backward
23
24        return out
25
26    def __mul__(self, other):
27        other = other if isinstance(other, Value) else Value(other)
28        out = Value(self.data * other.data, (self, other), '*')
29
30        def _backward():
31            self.grad += other.data * out.grad
32            other.grad += self.data * out.grad
33        out._backward = _backward
34
35        return out
36
37    def tanh(self):
38        x = self.data
39        t = (math.exp(2*x) - 1)/(math.exp(2*x) + 1)
40        out = Value(t, (self, ), 'tanh')
41
42        def _backward():
43            self.grad += (1 - t**2) * out.grad
44        out._backward = _backward
45
```

```
return out
```

This was a huge "aha" moment for me. By overloading the addition and multiplication operators, I could just write $w \cdot x + b$ naturally in Python, and behind the scenes, my code was silently building a directed acyclic graph (DAG) of the mathematical expression.

3 Week 2: Backpropagation & Optimization Math

3.1 The Chain Rule and Gradients

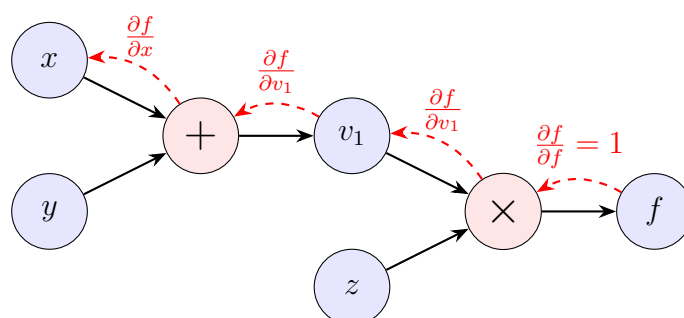
Week 2 was all about the backward pass. The central question of backpropagation is: "If I tweak this specific weight by a tiny amount, how much does the final loss change?"

To find this out, I derived the backpropagation equations by hand. It all boils down to the chain rule from calculus. If we have a sequence of operations $x \rightarrow y \rightarrow z$, the derivative of z with respect to x is:

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \cdot \frac{\partial y}{\partial x} \quad (2)$$

3.2 Directed Acyclic Graph (DOT-style Representation)

To visualize how the `Value` object creates a computational graph, I constructed a simple expression $f = (x + y) \times z$. When computed, it forms a directed acyclic graph (DAG) where values flow forward, and gradients flow backward.



In the context of the `Value` object, every time an operation happens, I defined a `_backward()` function that calculates the local derivative and multiplies it by the global derivative (`out.grad`) flowing backward from the node ahead of it.

3.3 Automating the Backward Pass (Topological Sort)

Doing this manually node-by-node works for one neuron, but it's impossible for a network of thousands of parameters. To automate this, I implemented a topological sort. This algorithm orders all the nodes in the graph so that we always calculate the gradient of a parent node before passing it backward to its children.

```
1 def backward(self):
2     topo = []
3     visited = set()
```

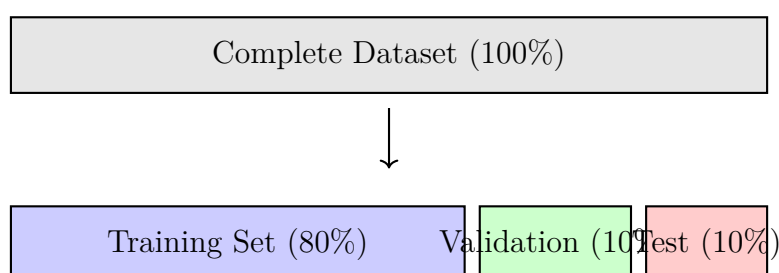
```
4     def build_topo(v):
5         if v not in visited:
6             visited.add(v)
7             for child in v._prev:
8                 build_topo(child)
9             topo.append(v)
10    build_topo(self)
11
12    self.grad = 1.0
13    for node in reversed(topo):
14        node._backward()
```


4 Week 3: Evaluation and Implementation Basics

4.1 Bias, Variance, and Data Splitting

After getting the raw math working, I needed to learn how to evaluate models properly. Before Week 3, I assumed that if a model gets 99% accuracy on the data you feed it, it's a good model. I learned this is actually a massive red flag for overfitting.

To test this practically, I took a tiny dummy dataset and manually split it on paper into a Training set, Validation set, and Test set to understand exactly how we score a model's performance.



4.2 Evaluation Metrics & Loss Functions

I also dug into Cross-Entropy versus Mean Squared Error (MSE). MSE is great for predicting continuous numbers, but awful for classification. For classification, we want the model to output a probability distribution, which is why we use Softmax combined with Cross-Entropy Loss.

I calculated evaluation metrics (Accuracy, Precision, Recall, and F1-Score) on a small confusion matrix by hand. It showed me why accuracy is useless on imbalanced datasets.

	Predicted Pos	Predicted Neg
Actual Pos	True Positive (TP)	False Negative (FN)
Actual Neg	False Positive (FP)	True Negative (TN)

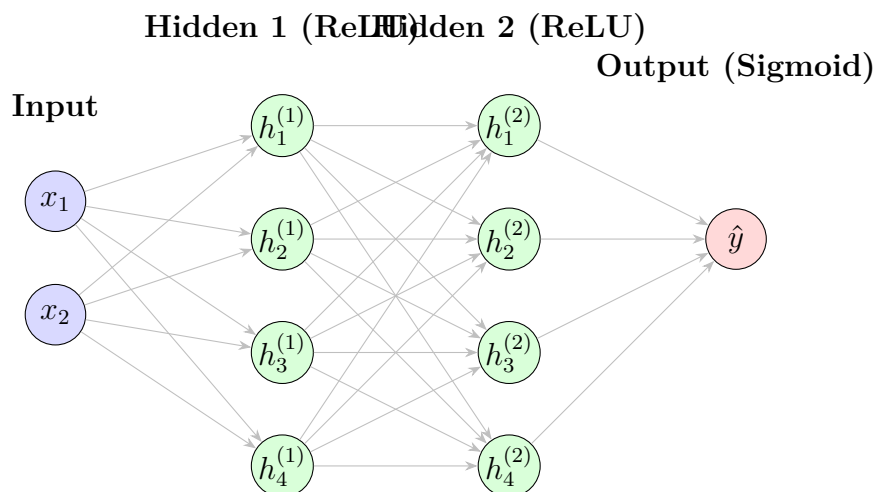
5 Week 4: Ground-Up Implementation (Midterm Project)

The highlight of the midterm checkpoint was shifting away from the pedagogical scalar `Value` class to a highly optimized, fully vectorized Multi-Layer Perceptron (MLP) built entirely from scratch using only `NumPy`.

5.1 MLP Architecture Design

The objective was to classify a complex non-linear dataset (like a 2D spiral). A single linear layer cannot solve this, so I designed an architecture with two hidden layers using ReLU activations, and a final output layer using a Sigmoid activation for binary classification.

Here is the DOT-style representation of the fully-connected architecture I implemented:



This network can be mathematically summarized as:

$$\begin{aligned} Z^{[1]} &= XW^{[1]} + b^{[1]}, & A^{[1]} &= \max(0, Z^{[1]}) \\ Z^{[2]} &= A^{[1]}W^{[2]} + b^{[2]}, & A^{[2]} &= \max(0, Z^{[2]}) \\ Z^{[3]} &= A^{[2]}W^{[3]} + b^{[3]}, & \hat{Y} &= \sigma(Z^{[3]}) \end{aligned}$$

5.2 Translating Math to Matrix Operations in NumPy

To make the training fast, I batched my inputs. The forward pass is highly optimized using matrix multiplications:

```

1 class NumPyMLP:
2     def __init__(self, input_dim, hidden_dim):
3         # He Initialization for ReLU layers
4         self.W1 = np.random.randn(input_dim, hidden_dim) * np.sqrt(2. /
5         input_dim)
6         self.b1 = np.zeros((1, hidden_dim))
7         self.W2 = np.random.randn(hidden_dim, hidden_dim) * np.sqrt(2.
8         / hidden_dim)
9         self.b2 = np.zeros((1, hidden_dim))
10        self.W3 = np.random.randn(hidden_dim, 1) * np.sqrt(1. /
11        hidden_dim)
12        self.b3 = np.zeros((1, 1))
13
14    def relu(self, Z):
15        return np.maximum(0, Z)
16
17    def sigmoid(self, Z):
18        return 1 / (1 + np.exp(-Z))
19
20    def forward(self, X):
21        self.Z1 = np.dot(X, self.W1) + self.b1
22        self.A1 = self.relu(self.Z1)
23
24        self.Z2 = np.dot(self.A1, self.W2) + self.b2
25        self.A2 = self.relu(self.Z2)
26
27        self.Z3 = np.dot(self.A2, self.W3) + self.b3
28        self.A3 = self.sigmoid(self.Z3)
29
30        return self.A3

```

5.3 The Backward Pass & L2 Regularization

Writing the backward pass in NumPy required translating the chain rule into Jacobian matrices. Additionally, to stop the network from memorizing the noise (overfitting), I integrated L2 Regularization directly into the gradient calculations. The penalty λ forces weights to remain small.

```

1     def backward(self, X, Y, learning_rate, lambda_l2=0.01):
2         m = X.shape[0] # batch size
3
4         # Derivative of Binary Cross Entropy + Sigmoid
5         dZ3 = self.A3 - Y
6         dW3 = (1/m) * np.dot(self.A2.T, dZ3) + (lambda_l2/m) * self.W3
7         db3 = (1/m) * np.sum(dZ3, axis=0, keepdims=True)
8

```

```

9      # Backprop through Layer 2 (ReLU)
10     dA2 = np.dot(dZ3, self.W3.T)
11     dZ2 = dA2 * (self.Z2 > 0) # Derivative of ReLU
12     dW2 = (1/m) * np.dot(self.A1.T, dZ2) + (lambda_12/m) * self.W2
13     db2 = (1/m) * np.sum(dZ2, axis=0, keepdims=True)
14
15     # Backprop through Layer 1 (ReLU)
16     dA1 = np.dot(dZ2, self.W2.T)
17     dZ1 = dA1 * (self.Z1 > 0)
18     dW1 = (1/m) * np.dot(X.T, dZ1) + (lambda_12/m) * self.W1
19     db1 = (1/m) * np.sum(dZ1, axis=0, keepdims=True)
20
21     # Gradient Descent Updates
22     self.W3 -= learning_rate * dW3
23     self.b3 -= learning_rate * db3
24     self.W2 -= learning_rate * dW2
25     self.b2 -= learning_rate * db2
26     self.W1 -= learning_rate * dW1
27     self.b1 -= learning_rate * db1

```

5.4 Stabilizing Training with Batch Normalization

Another crucial implementation detail was Batch Normalization. Without it, deep networks suffer from internal covariate shift. I wrote a manual function to calculate the mean and variance across the mini-batch and standardize the activations before passing them to the ReLU function.

```

1 def batchnorm_forward(Z, gamma, beta, eps=1e-5):
2     mu = np.mean(Z, axis=0)
3     var = np.var(Z, axis=0)
4     Z_norm = (Z - mu) / np.sqrt(var + eps)
5     out = gamma * Z_norm + beta
6     cache = (Z, Z_norm, mu, var, gamma, beta, eps)
7     return out, cache

```

5.5 Midterm Results

I integrated the forward pass, backward pass, and regularization into a training loop, running it for 10,000 epochs. When I fed it a complex 2D spiral dataset, the basic model wildly overfit the data.

However, with L2 regularization active, the decision boundary smoothed out beautifully. When I graphed the loss curves, the difference was undeniable: the validation loss curve hugged the training loss tightly instead of diverging upward. This project solidified

my understanding of how all the disjointed mathematical concepts fit together into a single cohesive, functional architecture.

6 Conclusion

These first four weeks have been intense but incredibly rewarding. By intentionally avoiding high-level frameworks, I have built a rock-solid mental model of what deep learning actually is. I no longer view neural networks as magic boxes; they are just chains of differentiable matrix operations being optimized by calculus.

As I move into Week 5, I am fully prepared to transition into PyTorch. Because I have already implemented automatic differentiation, matrix operations, cross-entropy, and batch norm from scratch, using PyTorch will feel like using a calculator after doing long division by hand. My next major goal is to use this foundational knowledge to dissect and understand the Math of Self-Attention and the Transformer architecture.